

07 — Experience and Scenario Questions (STAR method)

The STAR method

Letter	Meaning	How long
S — Situation	Where, which project, what was the problem	1–2 lines
T — Task	What was your responsibility	1 line
A — Action	What YOU did, step by step (most important part)	3–5 lines
R — Result	Number or outcome, and what you learned	1–2 lines

Prepare these 10 stories well. One story can answer many different questions.

Story 1 — Slow query: 92 seconds to 300 ms

Use for: performance, problem solving, biggest technical achievement, debugging

- **S:** "At Atiya, QA auditors used a call-quality report. Loading one call took about 92 seconds, so they couldn't audit many calls."
 - **T:** "I had to make the report fast without changing the database used by other systems."
 - **A:** "I checked the execution plan. The SQL Server view used OUTER APPLY, which ran a sub-query for every row. I bypassed the view and queried the base tables directly, used UNION ALL for the two call legs, and added indexes on the filter and join columns. I also added query timeouts so a slow query couldn't hang the server."
 - **R:** "Load time dropped from about 92 seconds to 200–350 milliseconds. QA could audit many more calls per day. I learned to always read the execution plan before guessing."
-

Story 2 — App freezing in production (event loop blocked)

Use for: production issue, debugging under pressure, Node.js internals

- **S:** "In ViewReports, sometimes the whole app froze for all users, and IIS recycled the process."
 - **T:** "Find the cause and stop the freezes."
 - **A:** "From the logs, freezes matched big date-range reports. The SQLite driver is synchronous, so a heavy query blocked the Node event loop. I added a date-range guard on heavy reports, moved index building to a worker thread, moved the data sync to a separate BullMQ worker with concurrency 1, and cached results keyed by the sync timestamp."
 - **R:** "No more freezes, and reports load in milliseconds from cache. I learned never to do heavy synchronous work on the main thread."
-

Story 3 — Legacy migration with 460+ tables

Use for: biggest achievement, smart work, ownership

- **S:** "The old CRM was ASP.NET WebForms with 460+ tables in two SQL Server databases."
 - **T:** "Rebuild it in Node.js + React with the same behaviour, without changing the database."
 - **A:** "I read the legacy C# code to understand business rules and made a parity checklist. Instead of hand-writing models, I wrote a script that parsed the SQL schema and generated 463 model files, plus a script to scaffold controllers and routes. Then I built 47 API modules on a layered structure and tested each screen against the old system."
 - **R:** "The new CRM went live on IIS and agents use it daily inside the dialer, with no change in their workflow. Processing time improved by about 30%."
-

Story 4 — Data was always empty (wrong data source)

Use for: mistake you learned from, debugging, attention to detail

- **S:** "My CallDropAutoDial job ran every 30 minutes to find dropped calls, but it kept finding zero."
 - **T:** "Find why the list was empty."
 - **A:** "I ran the query manually with different time windows and checked when rows were inserted. The child table I used was filled by an ETL job only once an hour, so the last 30 minutes were always empty. I switched to the live parent table, tested with DRY_RUN mode first, and documented the reason."
 - **R:** "The job started finding the right calls and pushing them for call-back. I learned to always check how and when data is written before building on it."
-

Story 5 — Webhook race condition

Use for: tricky bug, real-time systems, reliability

- **S:** "In the WhatsApp platform, sometimes the delivery status for a message was lost."
 - **T:** "Make status updates reliable."
 - **A:** "I found that Exotel's delivery webhook sometimes arrived before my code finished saving the outgoing message, so the update found nothing. I changed the handler to upsert by the Exotel message ID. Now whichever arrives first creates the record, and the other updates it. This also made duplicate webhooks safe."
 - **R:** "No more lost statuses or duplicate messages."
-

Story 6 — Leading TorahAnytime

Use for: leadership, teamwork, mentoring, client handling

- **S:** "At Morpheme, I was given responsibility for TorahAnytime, a live platform for an Israeli client."
- **T:** "Manage development from our side and deliver features safely."
- **A:** "I broke client requests into tickets, assigned them based on each developer's strength, reviewed every pull request for logic, performance and security, merged and deployed to production, and verified

after deploy. When a developer got stuck, I paired with them. I also built features myself — likes, editable playlists, legacy speakers and the most-viewed filter."

- **R:** "Releases became regular and predictable, and the client trusted us with more work. I learned how to review code and communicate with an international client."
-

Story 7 — Saying no to a risky request

Use for: conflict, disagreement with manager, communication

- **S:** "There was a suggestion to automatically retry failed uploads to the dialer."
 - **T:** "Decide the right retry behaviour."
 - **A:** "I explained that if the first upload actually succeeded but the response failed, a retry would make the dialer call the same customer twice — a bad customer experience. I proposed logging failures and a manual re-trigger with a preview instead. I showed it with an example and documented the decision."
 - **R:** "The team agreed. No double calls to customers. I learned to disagree with data and offer an alternative, not just say no."
-

Story 8 — Learning something new quickly

Use for: adaptability, learning ability

- **S:** "At Morpheme I had to build serverless APIs on AWS Lambda with TypeScript, which I hadn't used before."
 - **T:** "Deliver working APIs within the sprint."
 - **A:** "I read the AWS docs, built a small test function first, learned serverless YAML, then built the CRUD APIs and a PDF-to-image function. I load tested with JMeter and optimised cold starts by reducing bundle size and reusing connections."
 - **R:** "Delivered on time, and later I optimised Lambda functions on FamilyOne too. Now AWS is one of my strengths."
-

Story 9 — Tight deadline

Use for: pressure, time management

- **S:** "Management needed consultation forms for 16 different diseases in the CRM, quickly."
 - **T:** "Deliver all forms in a short time."
 - **A:** "Instead of building 16 separate screens, I designed one dynamic form engine driven by JSON schemas, with draft autosave and pre-fill from the last consultation. Each new disease became a JSON file instead of new code."
 - **R:** "All 16 forms went live on time, and adding a new form now takes minutes."
-

Story 10 — Deployment issue on IIS

Use for: DevOps, production support

- **S:** "After deploying a config change on IIS, the app was still running old behaviour."
- **T:** "Find why the change wasn't applied."
- **A:** "I checked the iisnode logs and found that iisnode only restarts when watched files change, and my `src` files weren't in the watched list. I fixed the `watchedFiles` setting in `web.config`, added startup logs that print the loaded config, and documented the app-pool restart command."
- **R:** "Deployments became reliable, and anyone can now verify which config is loaded."

Quick mapping: question → which story

Question	Story
Biggest technical challenge?	1 or 2
Biggest achievement?	3
A production issue you fixed?	2 or 10
A mistake you made?	4
A difficult bug?	5
Leadership / mentoring?	6
Conflict or disagreement?	7
Learning a new technology fast?	8
Working under a deadline?	9
Improving performance?	1

Scenario questions ("What would you do if...")

Your API is suddenly slow in production. What do you do?

"First check monitoring and logs: which endpoint, since when, error rate. Check recent deployments. Then check the database — slow query log, execution plan, missing index, locks. Check CPU and memory of the server and whether the event loop is blocked. Check third-party API latency. Fix the root cause — often an index or a cache. If it's urgent, roll back the last deploy first. After that, write a short RCA."

Production is down and your manager is on leave.

"Inform the team and stakeholders immediately. Check health endpoints and logs. If a recent deploy caused it, roll back. Restart services if needed. Once it's stable, find the root cause, fix it properly, and share a short report."

You get a requirement that isn't clear.

"I ask questions before coding — expected input, output, edge cases, and who will use it. I write a short summary and confirm it with the stakeholder. In my CRM work I used screenshots of the old system to confirm behaviour."

You can't finish a task by the deadline.

"I inform my manager early, not on the last day. I explain what's done, what's left and why, and suggest options — reduce scope, extend the date, or get help."

A teammate's code has a bug that reached production.

"Fix the issue first. Then discuss it privately and politely, focus on the process, and add a test or checklist so it doesn't happen again. No blaming."

You disagree with a senior's technical decision.

"I share my concern with data and an alternative. If they still decide differently, I respect the decision and support it fully."

You find a security issue in existing code.

"Report it to my lead immediately, assess the impact, fix it with priority, and rotate any exposed secrets."

How do you estimate a task?

"Break it into small parts — API, DB, UI, testing, deployment. Estimate each, add a buffer for unknowns and integration, and share the estimate with the assumptions."

How do you handle a client who keeps changing requirements?

"Understand why it's changing, document every change, explain the impact on timeline, and agree on priorities. Short demos help catch changes early."